C# Collections & Generics: Scenario-Based Coding Challenges

📊 Challenge 1: Hospital Patient Management System

Scenario

You're building a patient management system for a hospital. The hospital needs to track patients, their appointments, and medical records efficiently using appropriate collections.

Requirements

1. Create a `Patient` class with: `Id` (int), `Name` (string), `Age` (int), `Condition` (string)
2. Use a `Dictionary<int, Patient>` to store patients by ID
3. Use a `Queue<Patient>` for the appointment waiting list
4. Use a `List<string>` for each patient's medical history

Tasks

```csharp
// Task 1: Implement Patient class with proper encapsulation
public class Patient
{
    // TODO: Add properties with get/set accessors
    // TODO: Add constructor
}

// Task 2: Implement HospitalManager class
public class HospitalManager
{
    private Dictionary<int, Patient> _patients = new Dictionary<int, Patient>();
    private Queue<Patient> _appointmentQueue = new Queue<Patient>();

    // Add a new patient to the system
    public void RegisterPatient(int id, string name, int age, string condition)
    {
        // TODO: Create patient and add to dictionary
    }

    // Add patient to appointment queue
    public void ScheduleAppointment(int patientId)
    {
```

```csharp
        // TODO: Find patient and add to queue
    }

    // Process next appointment (remove from queue)
    public Patient ProcessNextAppointment()
    {
        // TODO: Return and remove next patient from queue
    }

    // Find patients with specific condition using LINQ
    public List<Patient> FindPatientsByCondition(string condition)
    {
        // TODO: Use LINQ to filter patients
    }
}
```

Test Cases

```csharp
HospitalManager manager = new HospitalManager();
manager.RegisterPatient(1, "John Doe", 45, "Hypertension");
manager.RegisterPatient(2, "Jane Smith", 32, "Diabetes");
manager.ScheduleAppointment(1);
manager.ScheduleAppointment(2);

var nextPatient = manager.ProcessNextAppointment();
Console.WriteLine(nextPatient.Name); // Should output: John Doe

var diabeticPatients = manager.FindPatientsByCondition("Diabetes");
Console.WriteLine(diabeticPatients.Count); // Should output: 1
```

📚 Challenge 2: Library Book Management System

Scenario

A library needs a system to manage books, track borrowing history, and maintain a catalog organized by genre using generics.

Requirements

1. Create a generic `Catalog<T>` class that can store any type of item
2. Use `List<Book>` for all books

3. Use `HashSet<string>` for tracking ISBNs (ensure uniqueness)
4. Use `SortedDictionary<string, List<Book>>` to organize books by genre

Tasks

```csharp
public class Book
{
    public string ISBN { get; set; }
    public string Title { get; set; }
    public string Author { get; set; }
    public string Genre { get; set; }
    public bool IsAvailable { get; set; }
}

// Generic catalog class
public class Catalog<T> where T : Book
{
    private List<T> _items = new List<T>();
    private HashSet<string> _isbnSet = new HashSet<string>();
    private SortedDictionary<string, List<T>> _genreIndex = new
SortedDictionary<string, List<T>>();

    // Add item with genre indexing
    public bool AddItem(T item)
    {
        // TODO: Check ISBN uniqueness, add to list and genre index
    }

    // Get books by genre using indexer
    public List<T> this[string genre]
    {
        get
        {
            // TODO: Return books by genre or empty list
        }
    }

    // Find books using LINQ and lambda expressions
    public IEnumerable<T> FindBooks(Func<T, bool> predicate)
    {
        // TODO: Use LINQ Where with predicate
    }
}
```

Test Cases

```csharp
```

```csharp
Catalog<Book> library = new Catalog<Book>();

Book book1 = new Book
{
    ISBN = "978-3-16-148410-0",
    Title = "C# Programming",
    Author = "John Sharp",
    Genre = "Programming"
};

library.AddItem(book1);

var programmingBooks = library["Programming"];
Console.WriteLine(programmingBooks.Count); // Should output: 1

var johnsBooks = library.FindBooks(b => b.Author.Contains("John"));
Console.WriteLine(johnsBooks.Count()); // Should output: 1
```

🛒 Challenge 3: E-commerce Shopping Cart

Scenario

Build a shopping cart system for an e-commerce platform that uses generics to handle different product types and applies discounts using delegates.

Requirements

1. Create generic `Product<T>` class
2. Use `Dictionary<T, int>` for cart items (product → quantity)
3. Implement discount calculation using `Func<T, double, double>`
4. Use LINQ for cart analytics

Tasks

```csharp
// Base product class
public abstract class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public double Price { get; set; }
}
```

```csharp
// Generic shopping cart
public class ShoppingCart<T> where T : Product
{
    private Dictionary<T, int> _cartItems = new Dictionary<T, int>();

    // Add product to cart
    public void AddToCart(T product, int quantity)
    {
        // TODO: Add or update quantity in dictionary
    }

    // Calculate total with discount delegate
    public double CalculateTotal(Func<T, double, double> discountCalculator = null)
    {
        double total = 0;
        foreach (var item in _cartItems)
        {
            double price = item.Key.Price * item.Value;
            if (discountCalculator != null)
                price = discountCalculator(item.Key, price);
            total += price;
        }
        return total;
    }

    // Get top N expensive items using LINQ
    public List<T> GetTopExpensiveItems(int n)
    {
        // TODO: Use LINQ OrderByDescending and Take
    }
}
```

Test Cases

```csharp
class Electronics : Product { }
class Clothing : Product { }

ShoppingCart<Electronics> cart = new ShoppingCart<Electronics>();

cart.AddToCart(new Electronics { Id = 1, Name = "Laptop", Price = 999.99 }, 1);
cart.AddToCart(new Electronics { Id = 2, Name = "Mouse", Price = 29.99 }, 2);

// Apply 10% discount for items over $100
double total = cart.CalculateTotal((product, price) =>
    price > 100 ? price * 0.9 : price);

Console.WriteLine($"Total: ${total:F2}");
```

```
var topItems = cart.GetTopExpensiveItems(1);
Console.WriteLine(topItems[0].Name); // Should output: Laptop
```

🏆 Challenge 4: Tournament Ranking System

Scenario

Create a tournament ranking system that uses multiple collection types to track teams, matches, and rankings.

Requirements

1. Use `SortedList<int, Team>` for rankings (auto-sorted by points)
2. Use `LinkedList<Match>` for match schedule
3. Use `Stack<Match>` for undo functionality
4. Implement custom comparer for sorting

Tasks

```csharp
public class Team : IComparable<Team>
{
    public string Name { get; set; }
    public int Points { get; set; }

    public int CompareTo(Team other)
    {
        // TODO: Compare by points descending, then by name
    }
}

public class Tournament
{
    private SortedList<int, Team> _rankings = new SortedList<int, Team>();
    private LinkedList<Match> _schedule = new LinkedList<Match>();
    private Stack<Match> _undoStack = new Stack<Match>();

    // Add match to schedule
    public void ScheduleMatch(Match match)
    {
        // TODO: Add to linked list
    }
```

```csharp
    // Record match result and update rankings
    public void RecordMatchResult(Match match, int team1Score, int team2Score)
    {
        _undoStack.Push(match.Clone());
        // TODO: Update team points and re-sort rankings
    }

    // Undo last match
    public void UndoLastMatch()
    {
        // TODO: Use stack to revert last match
    }

    // Get ranking position using binary search
    public int GetTeamRanking(Team team)
    {
        // TODO: Implement ranking lookup
    }
}
```

Test Cases

```csharp
Tournament tournament = new Tournament();
Team teamA = new Team { Name = "Team Alpha", Points = 0 };
Team teamB = new Team { Name = "Team Beta", Points = 0 };

tournament.ScheduleMatch(new Match(teamA, teamB));
tournament.RecordMatchResult(/* match */, 3, 1); // Team A wins

var rankings = tournament.GetRankings();
Console.WriteLine(rankings[0].Name); // Should output: Team Alpha

tournament.UndoLastMatch();
Console.WriteLine(teamA.Points); // Should output: 0 (back to original)
```

📈 Difficulty Progression

Level	Focus Area	Key Concepts
Beginner	Basic collections	List<T>, Dictionary<TKey,TValue>, foreach loops

Level	Focus Area	Key Concepts
Intermediate	LINQ & Delegates	Where, Select, OrderBy, lambda expressions
Advanced	Custom Collections	Sorted collections, custom comparers, generics constra
Expert	Performance & Thread Safety	Concurrent collections, IEnumerable vs IQueryable

💡 Learning Tips

1. **Start Simple**: Begin with List<T> and Dictionary<TKey,TValue> before advanced collections
2. **Practice LINQ**: Write the same query using both method syntax and query syntax
3. **Understand Trade-offs**: Know when to use List vs HashSet vs Dictionary
4. **Use Generics Wisely**: Create your own generic classes for reusable code

🎯 Next Steps

After mastering these collections challenges, consider:

1. **Thread-safe collections** (ConcurrentDictionary, BlockingCollection)
2. **Immutable collections** for functional programming
3. **Custom collection classes** implementing IEnumerable<T>
4. **Performance optimization** with Span<T> and Memory<T>

Would you like me to provide the complete solutions for any of these challenges, or would you prefer to work through them with hints?

More complex on the same

🚀 Advanced C# Collections & Generics: Complex Scenario Challenges

⚡ Challenge 1: Real-Time Stock Trading Platform

Scenario

Build a high-frequency trading system that processes thousands of orders per second with thread-safe collections, priority queues, and real-time analytics.

csharp

```csharp
// Complex Order Types with Generics
public interface IOrder<T> where T : IComparable<T>
{
    string OrderId { get; }
    T Instrument { get; }
    OrderSide Side { get; }
    decimal Price { get; }
    int Quantity { get; }
    DateTime Timestamp { get; }
    int Priority { get; } // 1-10, 1 is highest
}

// Thread-safe Order Book with Price-Time Priority
public class OrderBook<T> where T : IComparable<T>
{
    // Concurrent collections for thread safety
    private ConcurrentDictionary<string, IOrder<T>> _allOrders = new
ConcurrentDictionary<string, IOrder<T>>();
    private ConcurrentPriorityQueue<IOrder<T>> _buyOrders = new
ConcurrentPriorityQueue<IOrder<T>>(
        Comparer<IOrder<T>>.Create((a, b) =>
        {
            // Priority first, then price (higher better for buys), then time
            int priorityCompare = a.Priority.CompareTo(b.Priority);
            if (priorityCompare != 0) return -priorityCompare;

            int priceCompare = b.Price.CompareTo(a.Price); // Higher price first
            if (priceCompare != 0) return priceCompare;

            return a.Timestamp.CompareTo(b.Timestamp); // Earlier first
        }));

    private ConcurrentPriorityQueue<IOrder<T>> _sellOrders = new
ConcurrentPriorityQueue<IOrder<T>>(
        Comparer<IOrder<T>>.Create((a, b) =>
        {
            int priorityCompare = a.Priority.CompareTo(b.Priority);
            if (priorityCompare != 0) return -priorityCompare;

            int priceCompare = a.Price.CompareTo(b.Price); // Lower price first
            if (priceCompare != 0) return priceCompare;

            return a.Timestamp.CompareTo(b.Timestamp);
        }));

    // Market data stream processor
    private BufferBlock<MarketData<T>> _marketDataStream = new
BufferBlock<MarketData<T>>();

    // Real-time analytics
```

```
    private CircularBuffer<decimal> _priceHistory = new
CircularBuffer<decimal>(1000);
    private ConcurrentDictionary<TimeSpan, decimal> _volumeByTime = new
ConcurrentDictionary<TimeSpan, decimal>();

    public async Task ProcessOrderAsync(IOrder<T> order)
    {
        // TODO: Implement atomic order processing with matching engine
        // 1. Add to appropriate priority queue
        // 2. Attempt to match with opposite side
        // 3. Update market data stream
        // 4. Calculate analytics in real-time
        // 5. Handle partial fills
    }

    public IEnumerable<OrderMatch<T>> GetOrderMatches(int count)
    {
        // TODO: Use PLINQ for parallel processing of match history
        // Include complex join operations between buys and sells
    }

    public decimal CalculateVWAP(TimeSpan period)
    {
        // TODO: Calculate Volume Weighted Average Price using LINQ aggregation
        // with windowing over time-based partitions
    }
}
```

Advanced Requirements:

- Implement lock-free algorithms using `Interlocked` operations
- Use `Memory<T>` and `Span<T>` for high-performance price calculations
- Create custom `IEnumerable<T>` with lazy evaluation for market data streams
- Implement circuit breaker pattern for error handling

🔬 Challenge 2: Genetic Algorithm Framework with Generic Constraints

Scenario

Create a generic evolutionary computation framework that can evolve any type of solution using advanced generics and covariance/contravariance.

```csharp
// Complex generic constraints and variance
public interface IChromosome<TGene, TFitness> : IComparable<IChromosome<TGene,
TFitness>>
    where TFitness : IComparable<TFitness>
{
    IReadOnlyList<TGene> Genes { get; }
    TFitness Fitness { get; }
    IChromosome<TGene, TFitness> Crossover(IChromosome<TGene, TFitness> other);
    void Mutate(double mutationRate);
}

// Generic evolutionary algorithm with multiple strategy patterns
public class EvolutionaryAlgorithm<TGene, TFitness, TChromosome>
    where TChromosome : class, IChromosome<TGene, TFitness>
    where TFitness : struct, IComparable<TFitness>
{
    private Population<TGene, TFitness, TChromosome> _population;
    private SelectionStrategy<TGene, TFitness, TChromosome> _selectionStrategy;
    private CrossoverStrategy<TGene, TFitness, TChromosome> _crossoverStrategy;

    // Multiple specialized collections
    private AdaptiveSortedList<TFitness, TChromosome> _elitePool = new
AdaptiveSortedList<TFitness, TChromosome>();
    private ConcurrentBag<TChromosome> _offspring = new ConcurrentBag<TChromosome>();
    private GeneticDiversityCache<TGene> _diversityCache = new
GeneticDiversityCache<TGene>();

    // Covariant and contravariant interfaces
    public interface ISelectionStrategy<in TChromosome, out TSelected>
    {
        IEnumerable<TSelected> Select(IEnumerable<TChromosome> population, int
count);
    }

    public interface ICrossoverStrategy<in TParent, out TChild>
    {
        TChild Crossover(TParent parent1, TParent parent2);
    }

    public async Task<(TChromosome BestSolution, IEnumerable<EvolutionMetrics>
History)>
        EvolveAsync(EvolutionConfiguration config, CancellationToken
cancellationToken)
    {
        // TODO: Implement parallel evolution with Task.WhenAll
        // 1. Evaluate fitness in parallel using Parallel.ForEachAsync
        // 2. Apply multiple crossover strategies concurrently
        // 3. Use genetic diversity measures to avoid premature convergence
        // 4. Implement island model with migration between subpopulations
    }
```

```csharp
    // Advanced LINQ with custom aggregators
    public PopulationStatistics GetStatistics()
    {
        var stats = _population.Chromosomes
            .AsParallel()
            .WithDegreeOfParallelism(Environment.ProcessorCount)
            .Aggregate(
                () => new PopulationStatistics(), // Seed
                (acc, chromosome) => acc.Accumulate(chromosome), // Accumulator
                (acc1, acc2) => acc1.Combine(acc2), // Combiner for parallel
                final => final.Normalize()); // Final selector
        return stats;
    }
}
```

Complex Features:

- Custom generic constraints with multiple interfaces
- Covariant/contravariant interfaces for strategy patterns
- Parallel genetic operations with cancellation support
- Memory-efficient chromosome representations using `Span<T>`

🌐 Challenge 3: Distributed Cache with Consistent Hashing

Scenario

Implement a distributed in-memory cache with consistent hashing, replication, and partition tolerance using advanced concurrent collections.

```csharp
// Generic distributed cache with consistent hashing ring
public class DistributedCache<TKey, TValue> : IDisposable
    where TKey : IEquatable<TKey>
    where TValue : class
{
    private readonly SortedDictionary<uint, CacheNode> _hashRing = new
SortedDictionary<uint, CacheNode>();
    private readonly ConcurrentDictionary<TKey, CacheEntry<TValue>> _localCache = new
ConcurrentDictionary<TKey, CacheEntry<TValue>>();
    private readonly ConcurrentDictionary<string, RingPosition> _nodePositions = new
ConcurrentDictionary<string, RingPosition>();
    private readonly ReaderWriterLockSlim _ringLock = new
ReaderWriterLockSlim(LockRecursionPolicy.SupportsRecursion);
```

```
    // Virtual nodes for better distribution
    private const int VirtualNodesPerPhysicalNode = 200;

    private class CacheNode : IComparable<CacheNode>
    {
        public string NodeId { get; }
        public uint HashPosition { get; }
        public ConcurrentBag<CacheShard<TKey, TValue>> Shards { get; } = new
ConcurrentBag<CacheShard<TKey, TValue>>();

        public int CompareTo(CacheNode other) =>
HashPosition.CompareTo(other.HashPosition);
    }

    // Consistent hashing with replication
    public async Task<bool> SetAsync(TKey key, TValue value, TimeSpan? ttl = null,
int replicationFactor = 3)
    {
        // TODO: Implement consistent hashing algorithm
        // 1. Calculate key hash
        // 2. Find successor nodes on hash ring
        // 3. Replicate to N nodes (replicationFactor)
        // 4. Handle node failures with retry logic
        // 5. Update consistency hashes for conflict resolution
    }

    public async Task<CacheResult<TValue>> GetAsync(TKey key, bool readRepair =
false)
    {
        // TODO: Implement read-repair and eventual consistency
        // 1. Read from primary node
        // 2. If readRepair enabled, check replicas for consistency
        // 3. Repair inconsistencies automatically
        // 4. Handle stale reads with version vectors
    }

    // Dynamic ring rebalancing
    public async Task RebalanceRingAsync()
    {
        // TODO: Implement minimal key movement during rebalancing
        // Use consistent hashing algorithm to move only K/N keys
        // where K = keys on leaving node, N = total nodes
    }

    // Complex query support
    public IAsyncEnumerable<KeyValuePair<TKey, TValue>> QueryAsync(
        Expression<Func<KeyValuePair<TKey, TValue>, bool>> predicate,
        CancellationToken cancellationToken = default)
    {
        // TODO: Implement distributed query across all nodes
```

```
        // Combine results using priority merge from multiple sources
    }
}
```

Advanced Requirements:

- Implement CRDTs (Conflict-Free Replicated Data Types) for eventual consistency
- Use `Channel<T>` for asynchronous node communication
- Implement bloom filters for efficient key existence checks
- Create custom partitioners for data distribution

📊 Challenge 4: Time-Series Database Engine

Scenario

Build a high-performance time-series database engine with columnar storage, compression, and complex temporal queries.

```csharp
// Generic time-series with columnar storage
public class TimeSeriesDatabase<TTimestamp, TValue>
    where TTimestamp : IComparable<TTimestamp>, IEquatable<TTimestamp>
    where TValue : struct
{
    // Columnar storage using segmented collections
    private SegmentedList<TTimestamp> _timestamps = new
SegmentedList<TTimestamp>(segmentSize: 10000);
    private SegmentedList<TValue> _values = new SegmentedList<TValue>(segmentSize:
10000);
    private SegmentedBitArray _nullFlags = new SegmentedBitArray(segmentSize: 10000);

    // Indexes for fast lookups
    private IntervalTree<TTimestamp, int> _timeIndex = new IntervalTree<TTimestamp,
int>();
    private SortedDictionary<TValue, BitmapIndex> _valueIndex = new
SortedDictionary<TValue, BitmapIndex>();

    // Compression dictionaries for repeating values
    private Dictionary<TValue, int> _valueDictionary = new Dictionary<TValue, int>();
    private List<TValue> _reverseDictionary = new List<TValue>();

    // Write-optimized buffer
    private CircularBuffer<TimeSeriesPoint<TTimestamp, TValue>> _writeBuffer =
        new CircularBuffer<TimeSeriesPoint<TTimestamp, TValue>>(capacity: 1000);
```

```
    public async Task AppendAsync(IEnumerable<TimeSeriesPoint<TTimestamp, TValue>>
points)
    {
        // TODO: Implement buffered writes with batch compression
        // 1. Buffer points in write-optimized structure
        // 2. Apply delta encoding for timestamps
        // 3. Dictionary compress repeating values
        // 4. Update indexes incrementally
        // 5. Flush to columnar storage in segments
    }

    // Complex temporal queries with window functions
    public IEnumerable<WindowResult<TTimestamp, TAggregate>>
RollingWindow<TAggregate>(
        TTimestamp start,
        TTimestamp end,
        TimeSpan windowSize,
        TimeSpan step,
        Func<IEnumerable<TValue>, TAggregate> aggregator,
        WindowAlignment alignment = WindowAlignment.Center)
    {
        // TODO: Implement efficient window sliding with overlap
        // Use segmented iteration to avoid full materialization
        // Support tumbling, hopping, and sliding windows
    }

    // Pattern matching in time series
    public IEnumerable<PatternMatch<TTimestamp>> FindPatterns(
        IEnumerable<TValue> pattern,
        TimeSpan maxGap,
        double similarityThreshold = 0.9)
    {
        // TODO: Implement subsequence matching with DTW (Dynamic Time Warping)
        // Use early abandoning and lower bounding for performance
    }

    // Cross-correlation between multiple series
    public CorrelationMatrix CrossCorrelate(
        IEnumerable<string> seriesNames,
        TimeSpan maxLag,
        CorrelationMethod method = CorrelationMethod.Pearson)
    {
        // TODO: Implement parallel correlation computation
        // Handle different sampling frequencies with interpolation
    }
}
```

Complex Features:

- Columnar storage with segment-based compression
- Multiple index types (interval trees, bitmap indexes)
- Advanced window functions with custom aggregators
- Pattern matching with similarity measures
- Lock-free concurrent appends